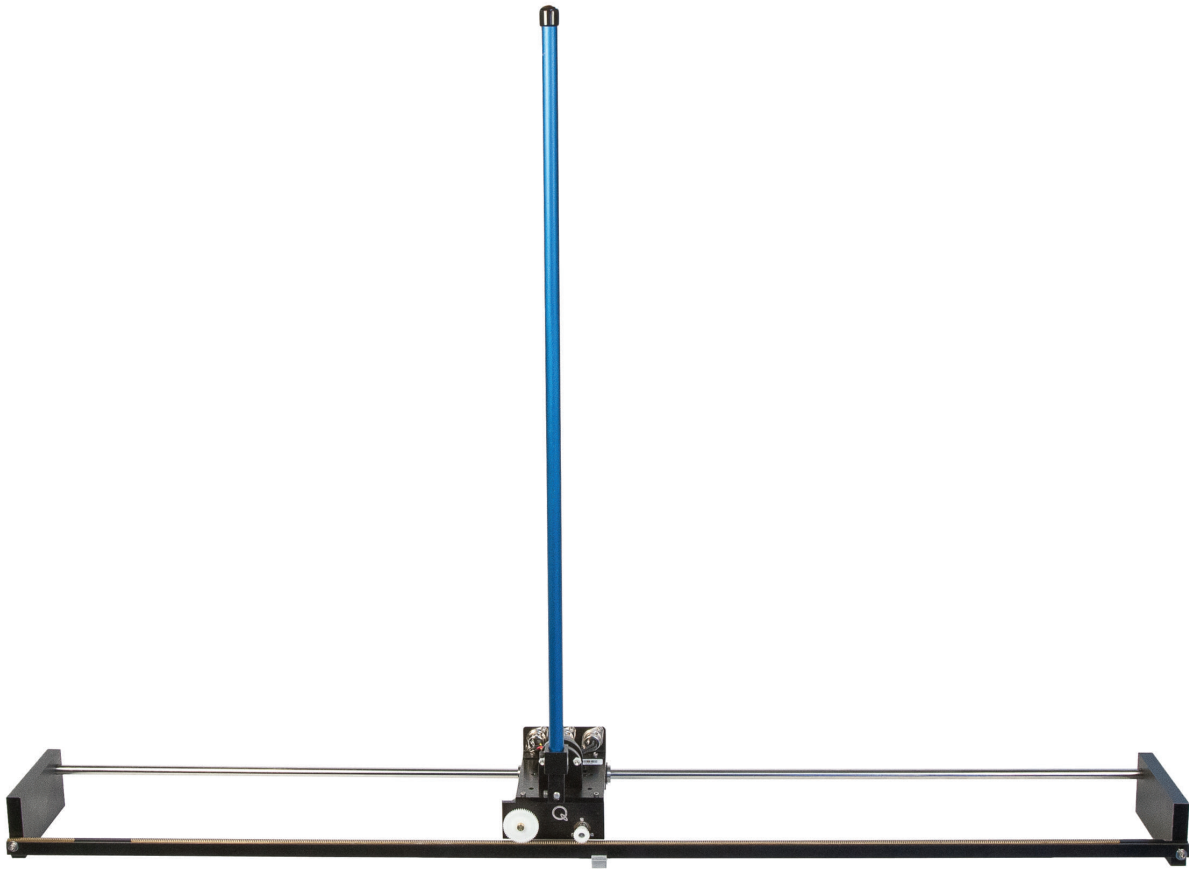


UCSB ECE Controls Lab TA Manual

By Max Crisafulli

September 2024 (Updated April 2026)



THIS DOCUMENT LIVES AT

<https://github.com/MaxCrisafulli/UCSB-ControlsLab-Manual-2024>

1 TA Manual for the UCSB ECE Controls Lab

This document is to act as a guide for TAs in setting up the Quanser inverted pendulum cart hardware in the UCSB ECE Controls Lab (Harold Frank Hall 3120A). This guide is an update of the existing manual:

<https://github.com/justinpearson/UCSB-Quanser-Inverted-Pendulum-Lab-Manual>

This updated guide will live and (ideally) be maintained at

<https://github.com/MaxCrisafulli/UCSB-ControlsLab-Manual-2024>

1.1 Hardware In The Lab

The ECE147ABC labs make use of the Quanser “Linear Servo Base Unit with Inverted Pendulum”. This manual will detail the setup of just the cart system, with motor voltage as input and linear position as output. There are 3 main components of the hardware setup, with each of which being explained below.

1.1.1 Q4 DAQ Board

The SIMULINK model that interfaces with the hardware interacts with the world via the Q4 HIL control card and DAQ (Data Acquisition Board). The control card is inside the lab PCs connected via PCIe and interfaces with the DAQ board via the two gray ribbon cables. Ensure that these are connected properly and that the red Status LED is lit, indicating that the board is powered.

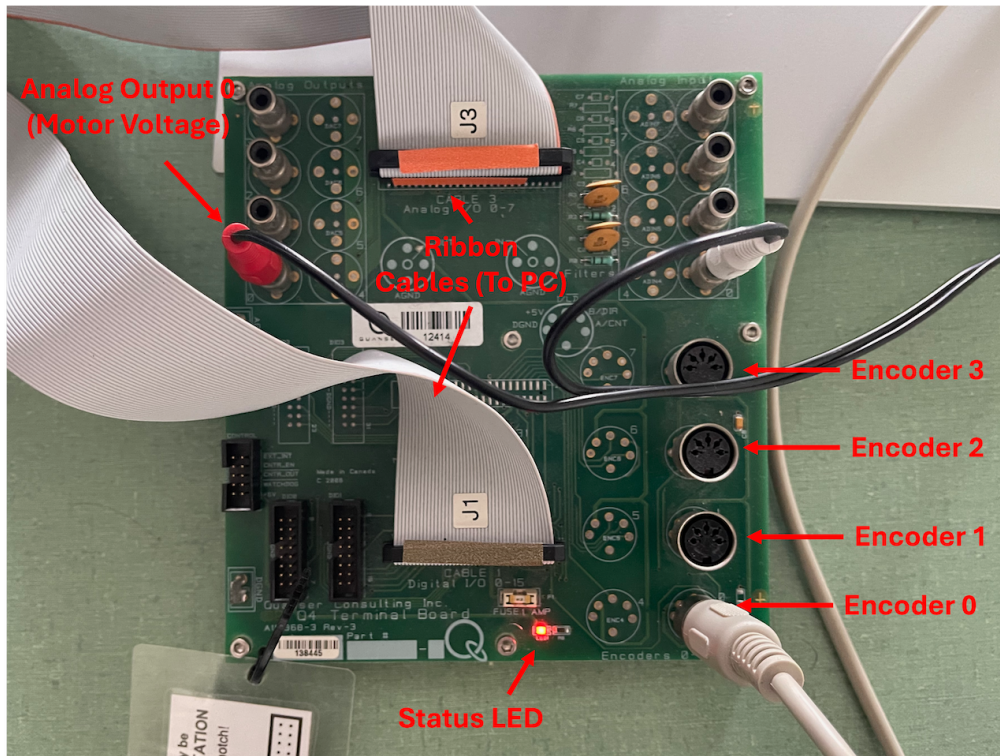


Figure 1: Quanser Q4 Terminal DAQ Board

The DAQ board uses the four encoder ports to read signals from the hardware. Generally, Encoder 0 is used for the linear position encoder. In the top left corner of the board are the Analog Outputs, Analog Output 0 is used to drive the amplifier box which ultimately drives the cart motor.

1.1.2 VoltPAQ-X1 Amplifier

The Quanser VoltPAQ-X1 Amplifier is powered from a standard wall socket, and turned on via a switch at the back of the box. Make sure that the green status LED in the center of the amplifier diagram is lit, indicating that the amplifier is powered on. Also check that the Amplifier Gain switch is set to 1x.

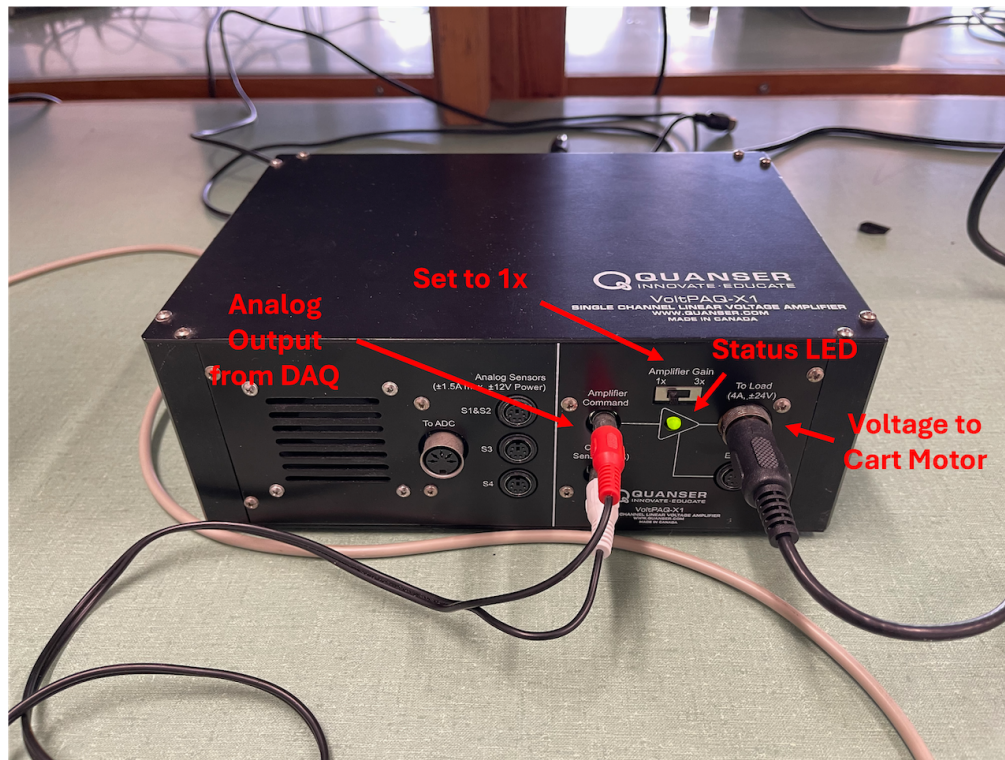


Figure 2: Quanser VoltPAQ-X1 Linear Voltage Amplifier

The red AV cable coming from Analog Output 0 on the DAQ board should be connected to the Amplifier Command (input) port. The white AV cable should be connected to Current Sense on the amplifier and Analog Input 0 on the DAQ board, although this is not necessary.

The output of the amplifier (To Load) is what drives the motor onboard the cart. Check that all the cable connections are correct and that the cable going to the motor has enough slack to allow the cart to move along the track.

1.1.3 Inverted Pendulum Cart System

The cart system has 2 encoder outputs and 1 analog voltage input. The positions of these ports may vary between the carts but in general they are as depicted in Figure 3. You should check what each of the ports connect to before plugging anything in.

The cart controls its position along the track with the smaller motor drive wheel, which is driven by the motor voltage coming from the amplifier. The larger position encoder wheel measures the relative position along the track from the time that the program was initialized in 'ticks'. The conversion ratio from ticks to inches is approximately 5000 ticks per 6 inches or 35000 ticks per 1m.

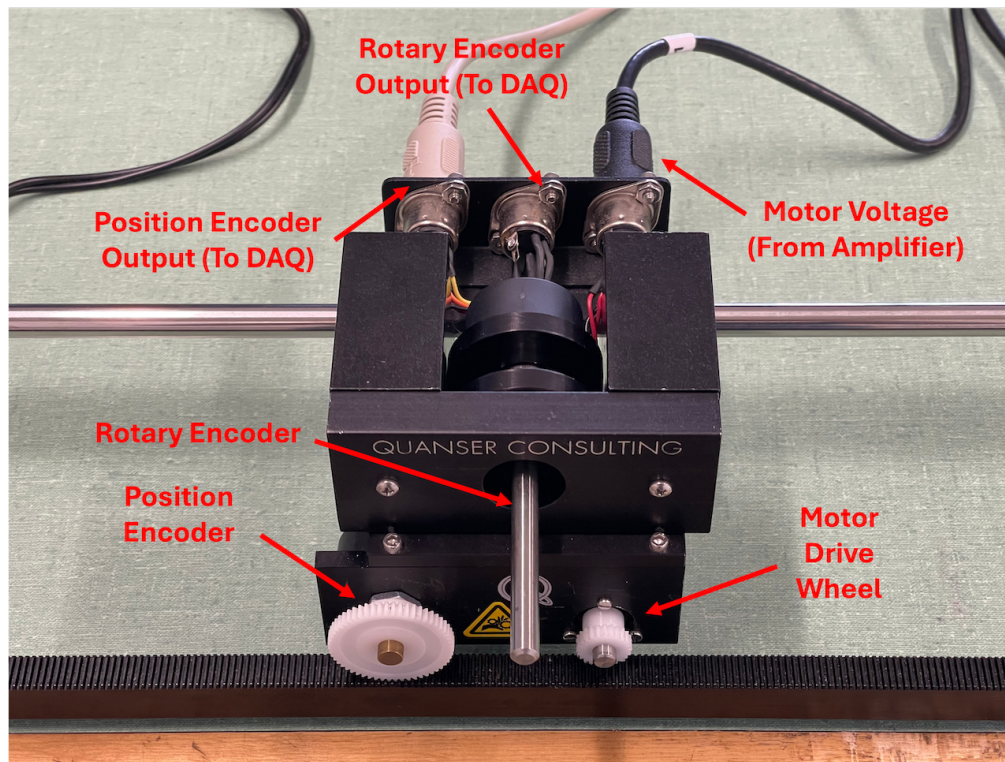


Figure 3: Quanser Linear Servo Base Unit (The 'Cart')

2 General Info

- If the DAQ board light is off, check that the ribbon cables going to/from the PC are connected properly. If there is still an issue then the (minimum 1 Amp) fuse above the LED may need replacing. Make everything is completely unplugged from the DAQ board before doing this. Spare fuses can be found in a toolbox in the gray cabinet at the back right of the lab.
- If the DAQ board is not connecting to the PC and the light is on try restarting the computer.
- Contact the IT people for computer problems, MATLAB issues, and getting TA/student login access:

`support@ece.ucsb.edu`

- Contact the ECE shop people for issues with the lab hardware:

`shop@ece.ucsb.edu`

3 Setup Instructions (ECE147A Lab 2)

3.1 Hardware

The *MultiQ-4 Terminal Board* (Figure 4) is a general purpose data acquisition and control board which has eight single ended analog inputs, eight analog outputs, sixteen bits of digital input, sixteen bits of digital output, three programmable timers, and eight encoder inputs decoded in quadrature. For the purposes of this lab, we will be primarily concerned with the analog outputs, the encoder inputs, and, to a lesser extent, the analog inputs.

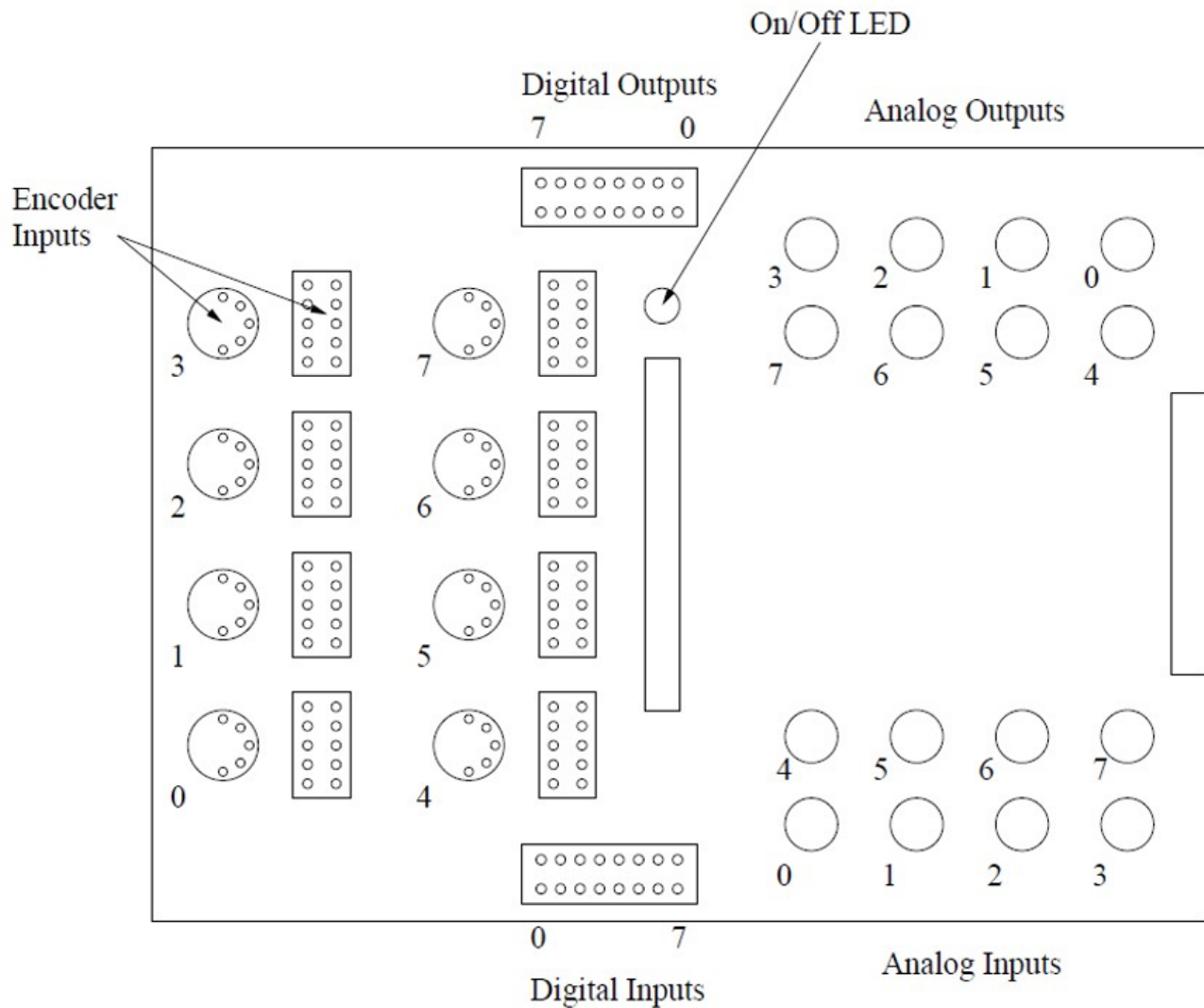


Figure 4: MultiQ-4 Terminal Board Schematic

Both the analog inputs and outputs are accessed via clearly labeled RCA jacks on the MultiQ-4 board and have a range of ± 5 volts. The encoder inputs are accessed via a 5 pin DIN socket and connect directly to the experimental equipment.

3.1.1 Hardware Setup

As a basic example, we will set up the system to read in the encoder from the motor cart and output a constant voltage to the motor.

1. You must use **MATLAB R2023a only**. The other versions will not work. Start **SIMULINK** and create a new model.
2. Choose **Modeling** → **Model Settings**
3. Set the following parameters
 - (a) Select **Solver** from the tree at the left
 - (b) Stop time: **inf** - or whatever T_f is desired (in seconds)
 - (c) Solver options, solver type: **Fixed-Step**
 - (d) Solver options, solver: **discrete** if there are no continuous states. Select a solver like **ode4** if you are using continuous states (like a continuous transfer function)
 - (e) Solver options, Fixed-step size (under Solver details): **0.001**
 - (f) The settings are illustrated:

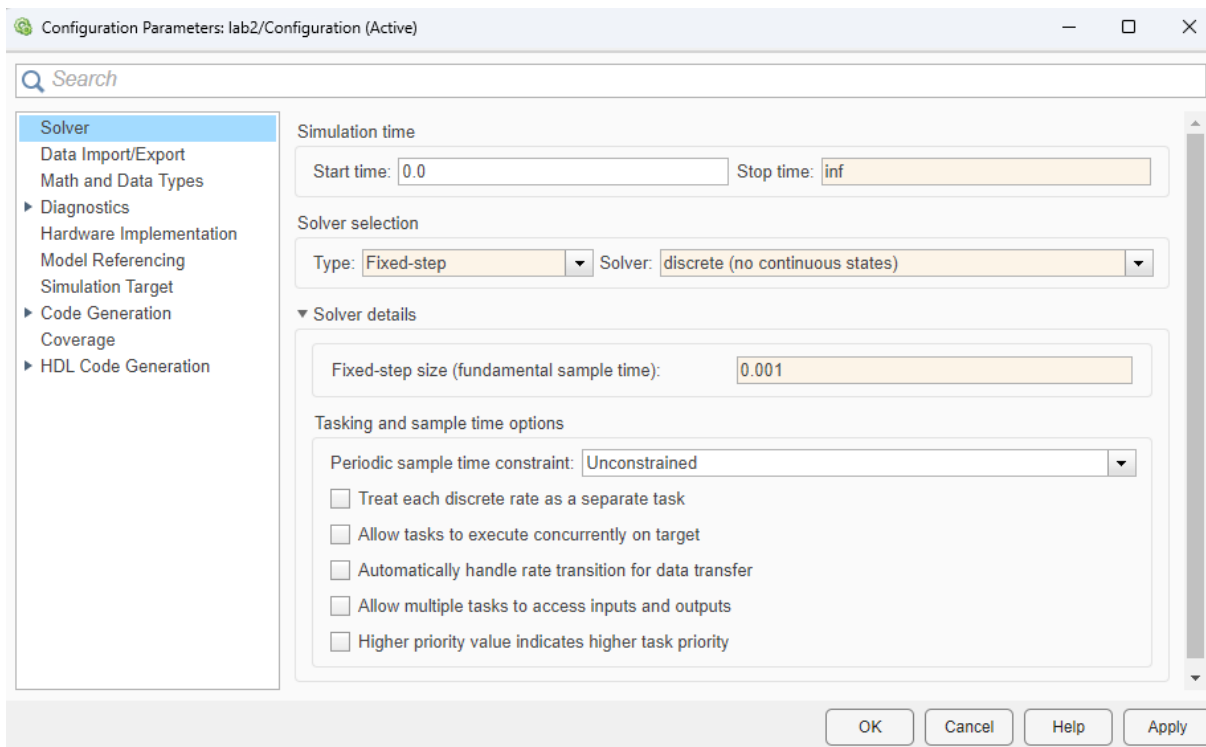


Figure 5: Configuration Parameters

- (g) Choose **Code Generation** in the tree on the left.
- (h) Ensure the System target file is **quarc_win64.tlc**. If not, select the **Browse** button and select it. The settings should look as follows.

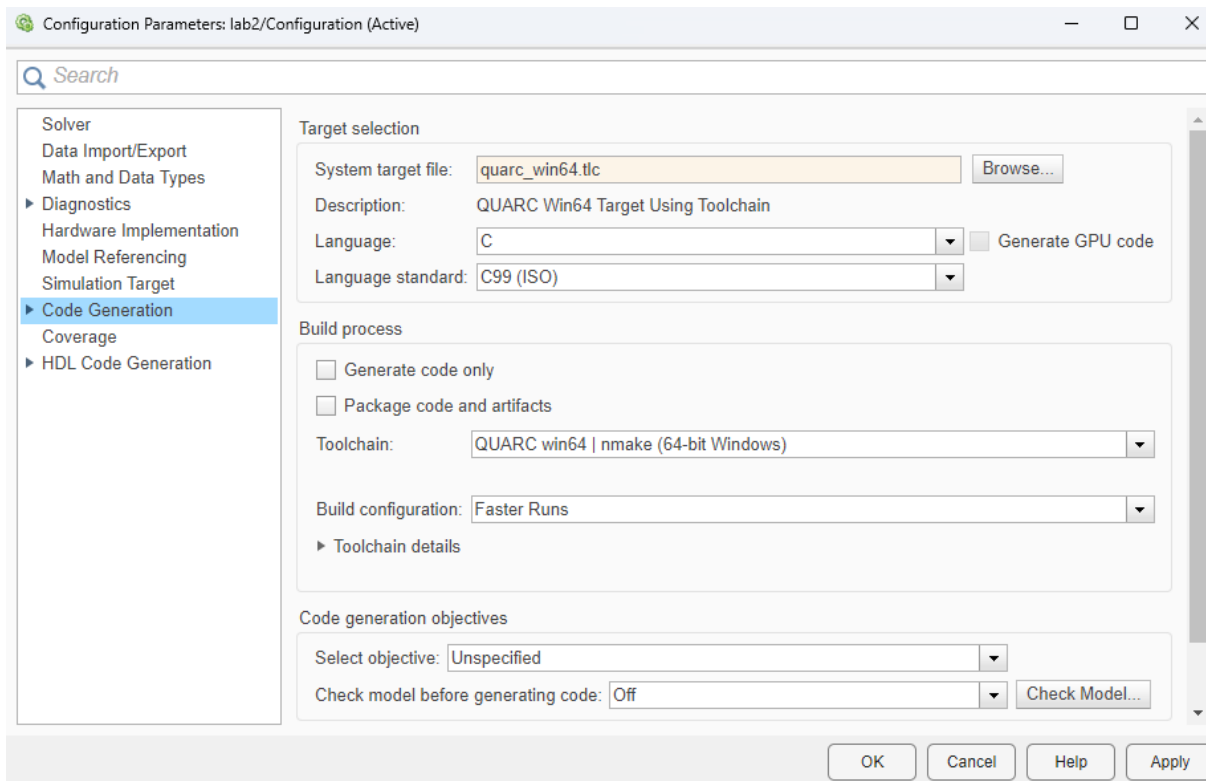


Figure 6: RTW Parameters

4. Select OK and return to the SIMULINK model.
5. In the SIMULINK Library Browser, find the HIL Initialize block. It is located in **QuaRC Targets** → **Data Acquisition** → **Generic** → **Configuration**. You can also search by double clicking the model background. Add the block into your model.
 - (a) *Aside:* HIL stands for *Hardware In the Loop*. It is a design technique in which the controller hardware is tested with a simulated plant. A flight simulator is an example; the pilot (controller) is connected to simulated inputs and outputs. The technique we are using in the lab is often called *Rapid Control Prototyping*, where a PC controller is connected to actual plant hardware. If you are simulating everything on a computer, like in the prelab, you are doing *offline simulation*.
6. Double click on the HIL Initialize block in your model. Change the Board type to **q4**. Make sure the Board Identifier is 0. Click Apply and Ok out of the block.

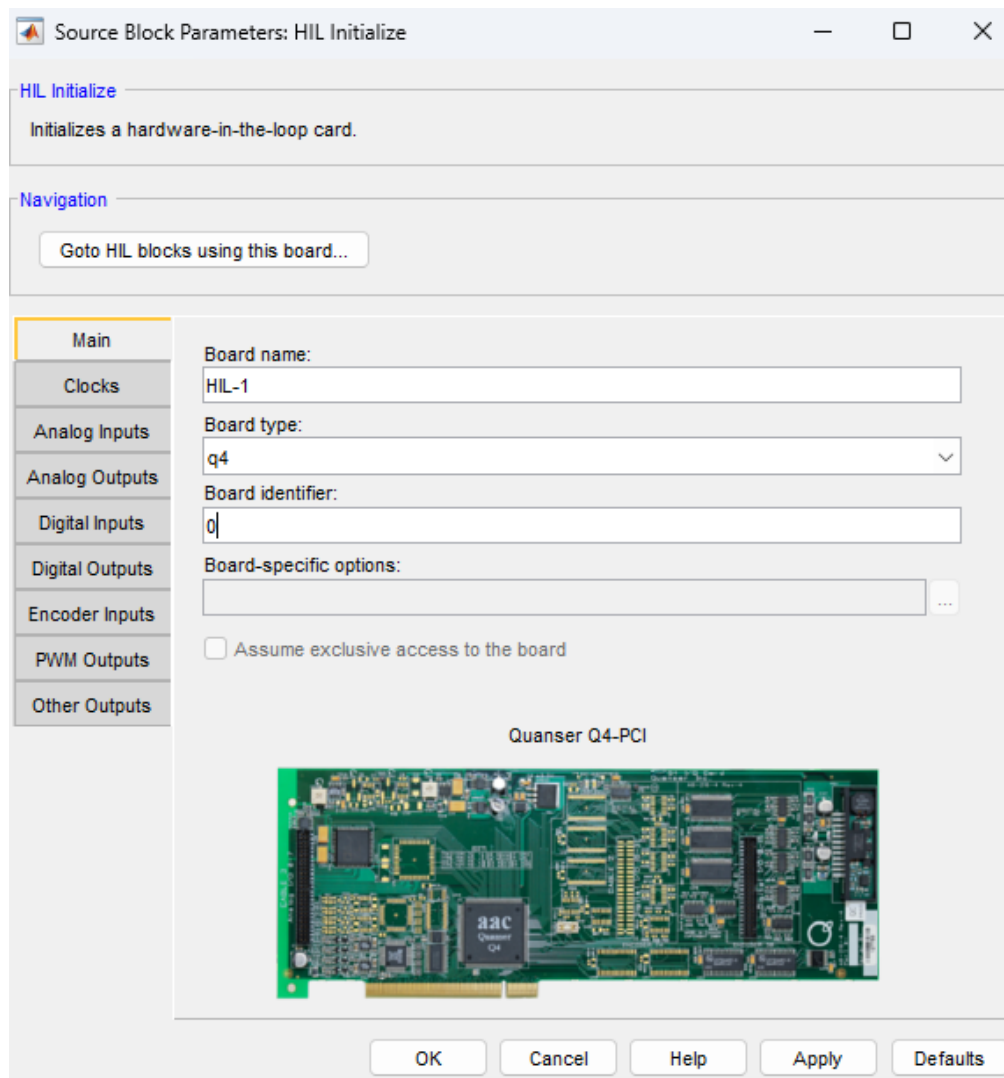


Figure 7: HIL Initialize Parameters

7. Add a HIL Write Analog block and a HIL Read Encoder block to your model. They are in QuaRC → Data Acquisition → Generic → Immediate I/O.
8. Connect these blocks as depicted in Figure 8. In this example, we place a simple constant output and a display for the encoder. *For the purposes of this example, do not output more than ± 2.5 volts to the cart to prevent uncontrolled motion.*

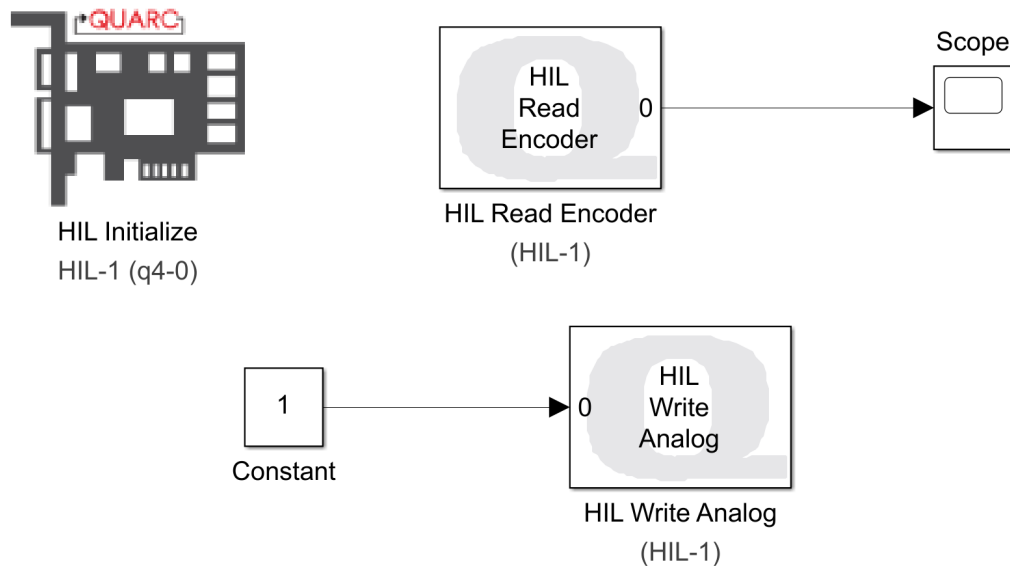


Figure 8: Simple Model

9. When your model is ready, save it. From the **QuaRC** menu, select **Build**. MATLAB will compile your SIMULINK model into native machine code. You can check the status of the build by clicking **View diagnostics** at the bottom of the SIMULINK window. *Every time you structurally change your model (add new blocks, rewire existing blocks) you will need to rebuild it or an error will occur.*
 - (a) *Tip:* You do not need to rebuild the model if you are changing constants, or blocks that take a constant, like the gain of the gain block. You can even change these constants while the model is running and the change will take effect immediately.
 - (b) *Tip:* You can change the constants in a transfer function while stopped or running such that they do not affect the dimensions of the state. That is, the number of poles and zeros must remain the same. Any other change requires a rebuild.
 - (c) *Use care when changing a running model! Be careful not to leave the model running by accident.*
10. Ask your TA to set up the cart for motion. It should be placed in the center of the track and the Quanser Amplifier switched off.
11. When the build is completed, go back to the model and choose **QuaRC** → **Monitor & Tune**. This downloads the compiled model to the QuaRC execution program and then model will run. The **Stop** button in the Quarc menu will stop the model as normal.
 - (a) *Troubleshooting Tip:* If the SIMULINK time is not running (at the bottom of the model window), the QuaRC execution environment may have crashed. Try logging out and logging back in or rebooting the computer.
 - (b) *Tip:* Suppose your model is running and MATLAB crashes. The model will continue to run. First, ensure the hardware is not damaged by lifting it off the track and then switching off the Amplifier.
12. As the model runs, observe that moving the cart will change the encoder output, as shown on the display. Lift the cart up from the track and turn on the Quanser Amplifier. Observe the motor motion.
13. Save your basic model to your network drive to use as a starting point for future designs.

3.1.2 Data Logging

In the Real Time Windows Target, data is logged by scope blocks. Like an oscilloscope, to start the data logging procedure, a trigger is needed to initiate the process. A simple approach is to define a dedicated scope block, with one trigger signal entering it. Put a step function with a step time of 0.001 seconds, which corresponds to one sample time. This set-up will save all but the very first data point. To record data, follow the procedure below.

1. Place a STEP function block from the SOURCES library and a SCOPE block from the SINKS library in your model.
2. Name the SCOPE block TRIGGER SCOPE so that it is easy to identify in a list.
3. Set the step time of the STEP block to the time you wish to begin saving data. Set the Initial Value and Final Value to 0 and 1 respectively.

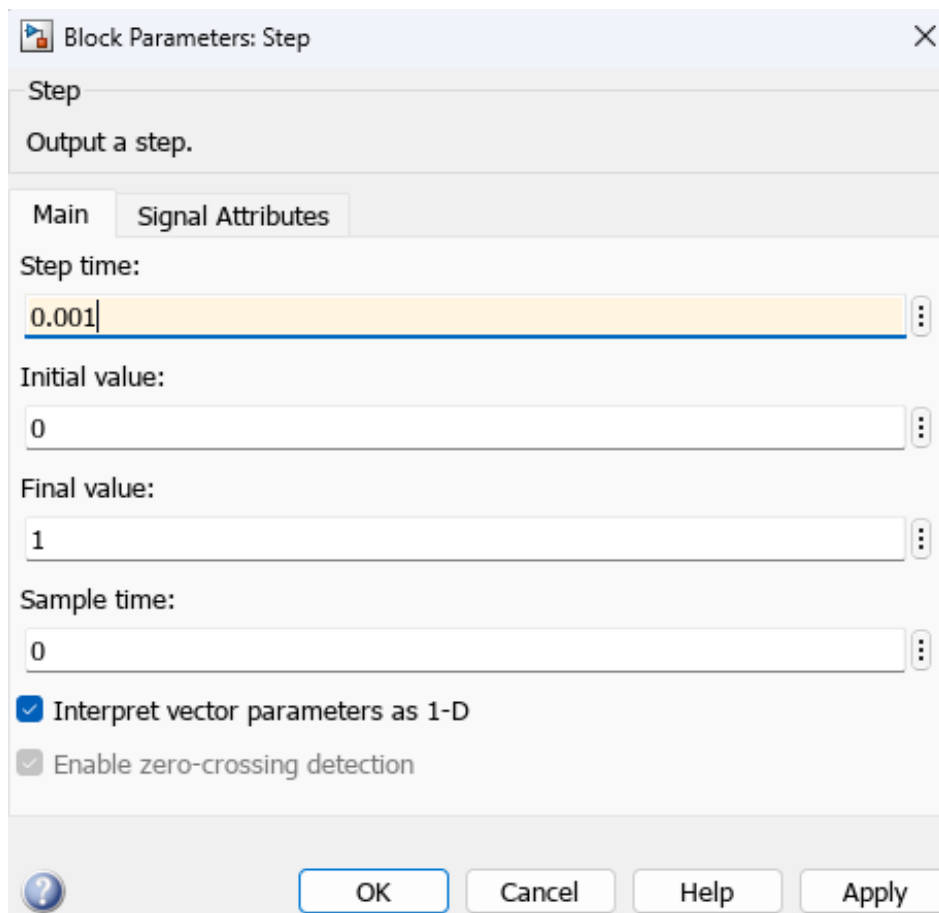


Figure 9: Step Function Block as a Trigger Signal

4. Go to the HARDWARE menu and select CONTROL PANEL.

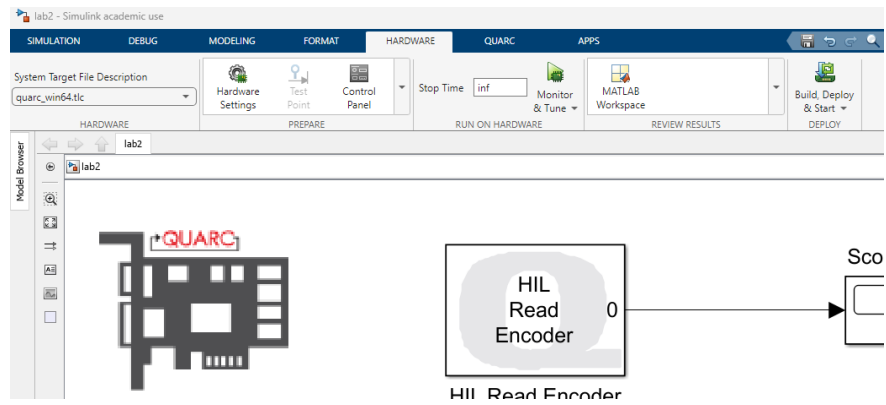


Figure 10: Hardware Control Panel

5. Click on the SIGNALS AND TRIGGERING button.
6. Select the TRIGGER SCOPE block from the list and click on the TRIGGER SIGNAL button. This selects the input to the scope block as the trigger signal.
7. In the TRIGGER area make sure that SOURCE = SIGNAL, MODE = NORMAL, DELAY = 0, and set DURATION = T_f/T_s . The duration is the number of samples allotted in memory for each signal being logged. If we have a simulation time of $T_f = 10$ seconds and our sampling time $T_s = 0.001$, then $10/0.001 = 10,000$ samples. If you actually run the model longer than T_f , then the data in memory will be overwritten when T_f/T_s samples have passed. Check Figure 11.
8. In the TRIGGER SIGNAL area make sure that DIRECTION = EITHER or RISING, and LEVEL = 0.5. Our STEP signal will transition from 0 to 1 at 0.001 seconds, and is a rising edge. When it crosses the value 0.5 this will initiate the signal logging.

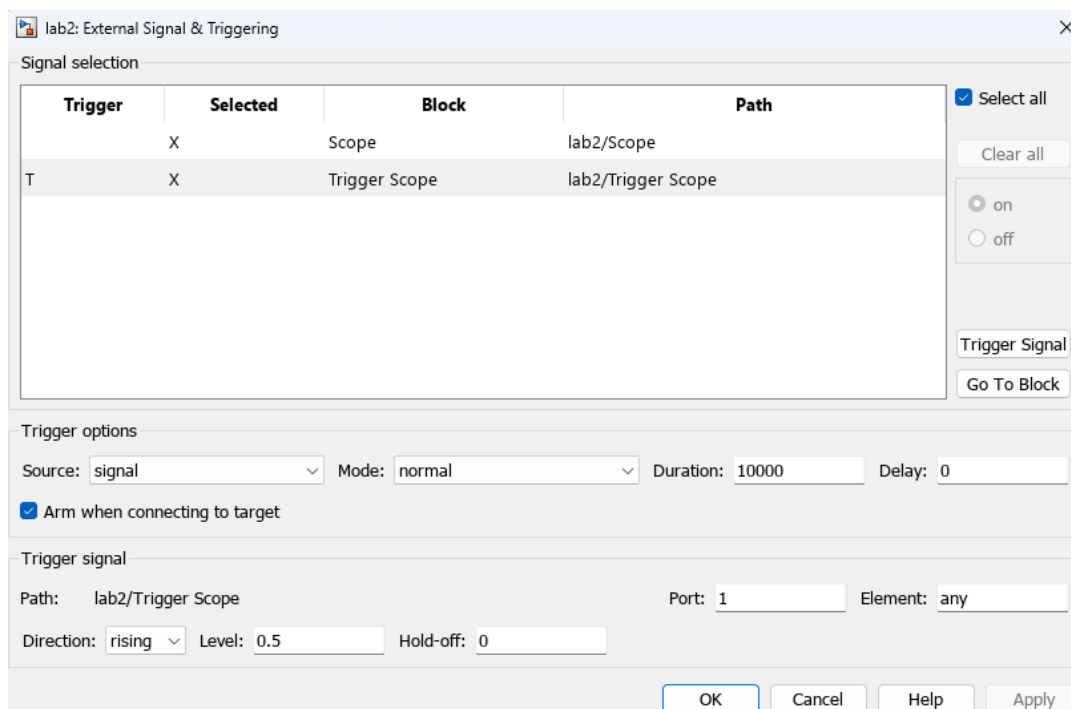


Figure 11: Configuring the Trigger Signal

9. Open all SCOPE blocks and configure them in the following way.

10. Click the SCOPE PROPERTIES Icon (gear icon), and select the LOGGING tab.
11. Make sure Limit data points to last is UNCHECKED.
12. If you wish to save the data gathered by the scope, CHECK the Log data to workspace box, and give the Variable name a unique and meaningful name for easy reference later. In the SAVE FORMAT menu choose the ARRAY option. Click Apply and Ok out of the menu.
13. *Hint:* Once you have set these parameters as you like, you can RIGHT CLICK on a scope block, drag and drop, and a copy of that block will be placed where you let go of the mouse. The new block will have all of the parameters of the previous, except the Variable name, that will be different and should be changed to something meaningful.

4 Demo Model

1. Generate the SIMULINK diagram shown in Figure 12. The Sine wave should have a relatively low magnitude of around 0.75 and a frequency of around 1 Hz.

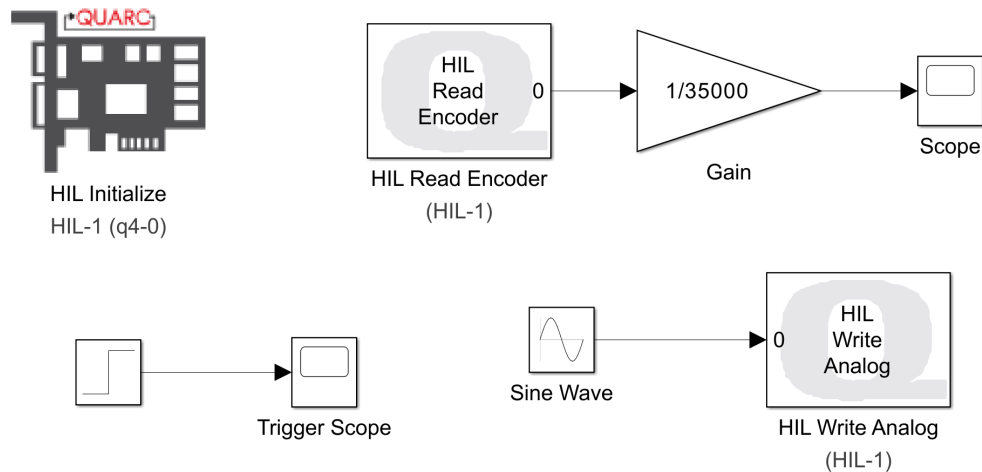


Figure 12: SIMULINK Loopback System

2. The output is an integer corresponding to counts. There is certain number of counts/revolution of the optical encoder's shaft. The gain seen in the figure is the 'conversion factor for encoder reading'. It should be $1/35000$, ask your TA to make sure. The $1/35000$ factor means that when encoder counts 35,000 in one direction it has moved 1 meter.
3. After creating the system diagram and saving it, build the model following steps written in Section 3.1.1. This generates the real-time code for the diagram. Wait until the compilation is complete. The SIMULINK window diagnostics panel displays the progress of the code generation task and, when it is complete, the following message appears:

```
▼ Build Summary

Top model targets built:

Model  Action                                     Rebuild Reason
=====
lab2    Code generated and compiled.  Generated code was out of date.

1 of 1 models built (0 models already up to date)
Build duration: 0h 0m 10.356s
```

Figure 13: Successful Code Generation Message

4. Check the necessary wires:
 - (a) Connect Analog Output 0 of the MultiQ-4 board to the Quanser Amplifier “Amplifier Command” via the red RCA cable.
 - (b) Connect the Quanser Amplifier “To Load” (6 pin DIN) to the motor cart’s motor (4 pin DIN).
 - (c) Connect the optical encoder which measures cart position (5 pin DIN) to MultiQ-4 Encoder Input 0 (5 pin DIN).
5. Make sure the Amplifier Gain is set to 1x. Ask for the TA’s permission to turn the Quanser Amplifier on. In unexpected situations, turn it off. Place the motor cart in the center of the track.
6. You should be able to run the model by clicking MONITOR & TUNE. Do not move the cursor. Press STOP immediately if needed.

Configuring the model to work with DAQ

The image displays three screenshots from the MATLAB/Simulink environment, illustrating the configuration of a hardware-in-the-loop (HIL) model.

Top Left: Source Block Parameters: HIL Initialize
This dialog box is used to configure the HIL Initialize block. The "Board name" is set to "HIL-1" and the "Board type" is set to "q4". The "Board identifier" is set to "0". The "Board-specific options" section is expanded, showing "Assume exclusive access to the board" as an option. The "Defaults" button is highlighted with a red box.

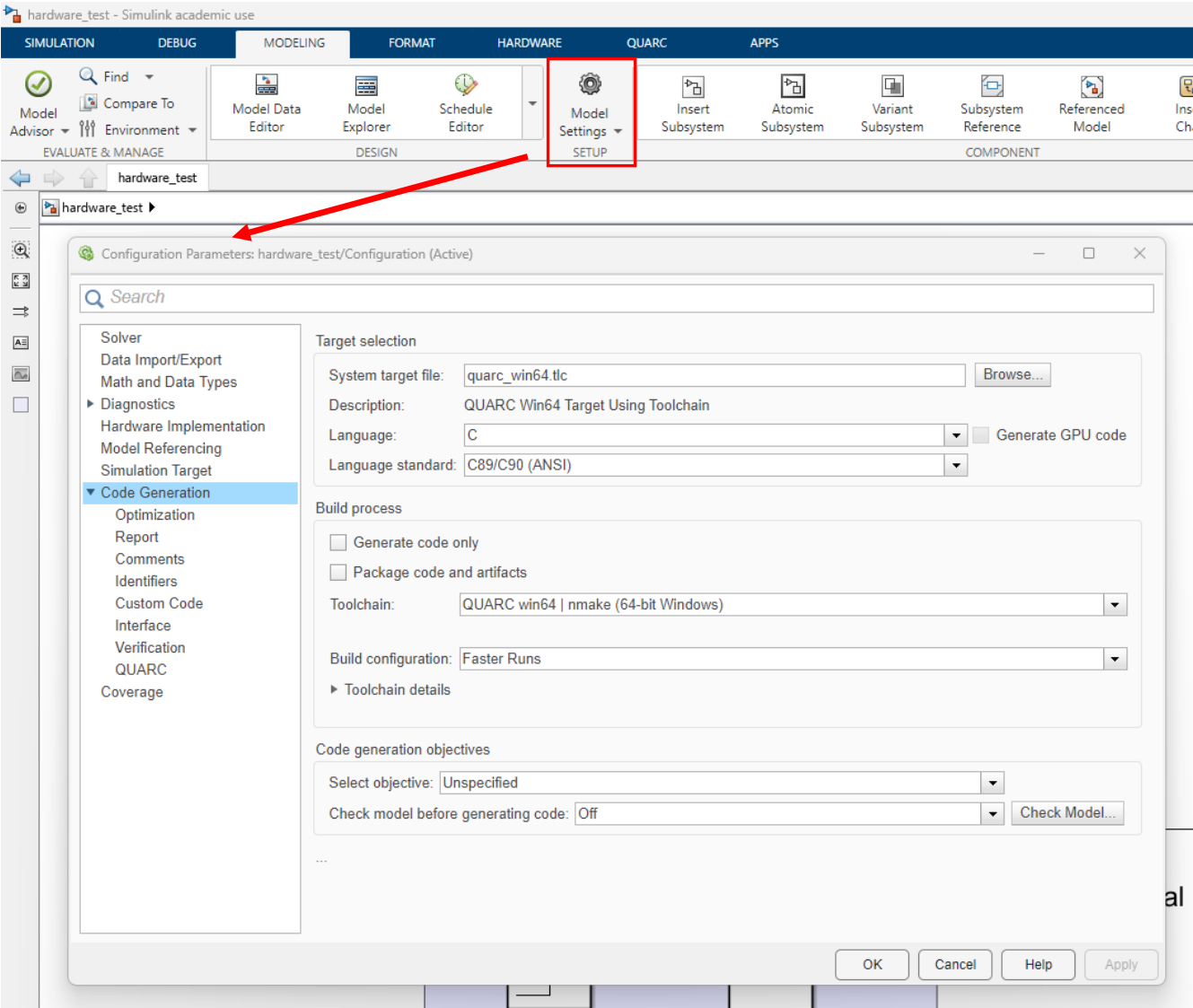
Top Right: hardware_test - Simulink academic use
This screenshot shows the main Simulink model window. The "HIL Configuration" block is visible, which contains the "HIL-1 (q4-0)" block. The "HIL-1 (q4-0)" block is highlighted with a red box. The "HIL Configuration" block also contains a "Trigger Scope" and a "ScopeTemplate" block.

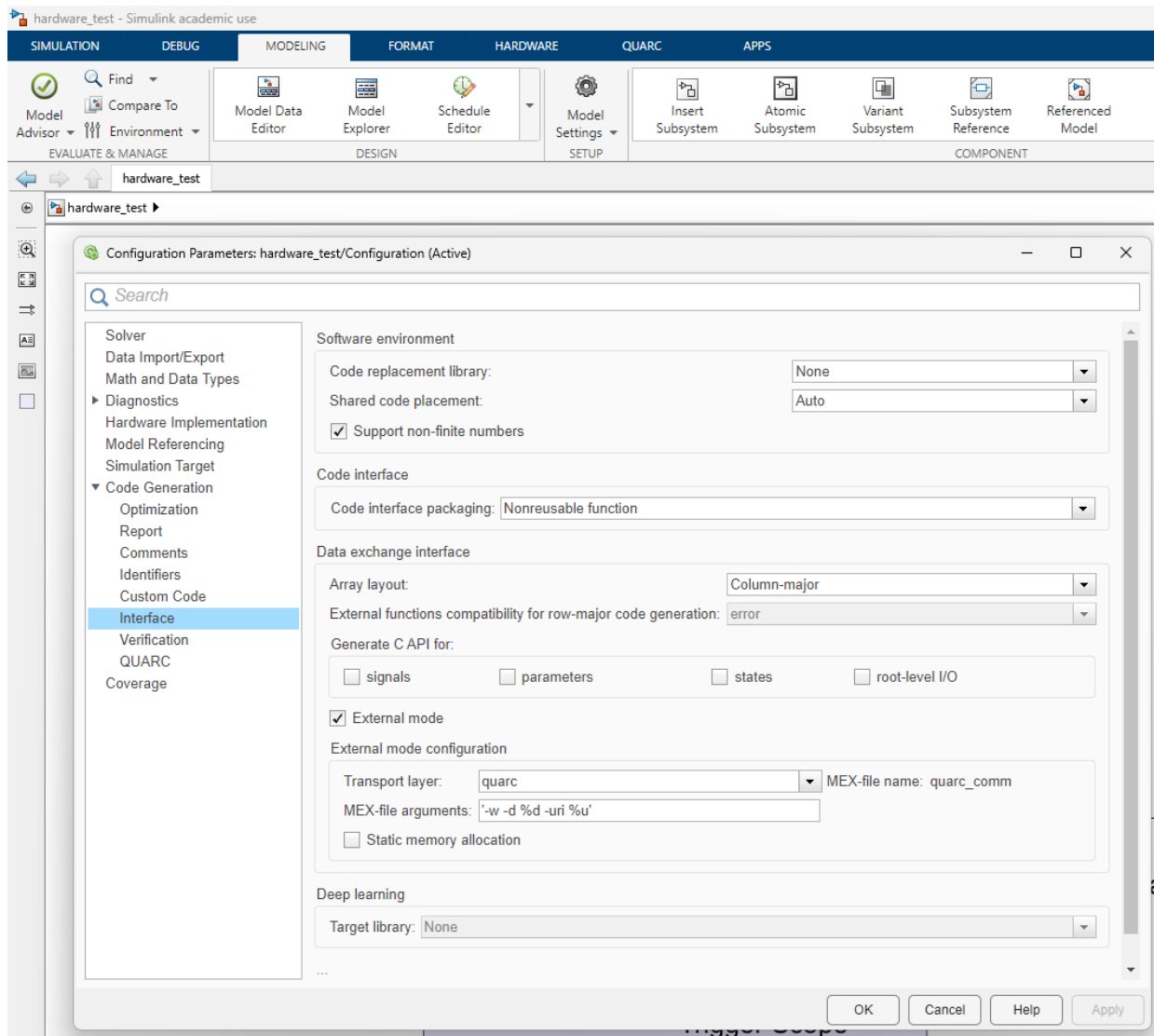
Bottom Left: Simulink Library Browser
This screenshot shows the Simulink Library Browser. The "QUARC Targets/Data Acquisition/Generic/Configuration" section is expanded, showing the "HIL Initialize" block. The "HIL Initialize" block is highlighted with a red box.

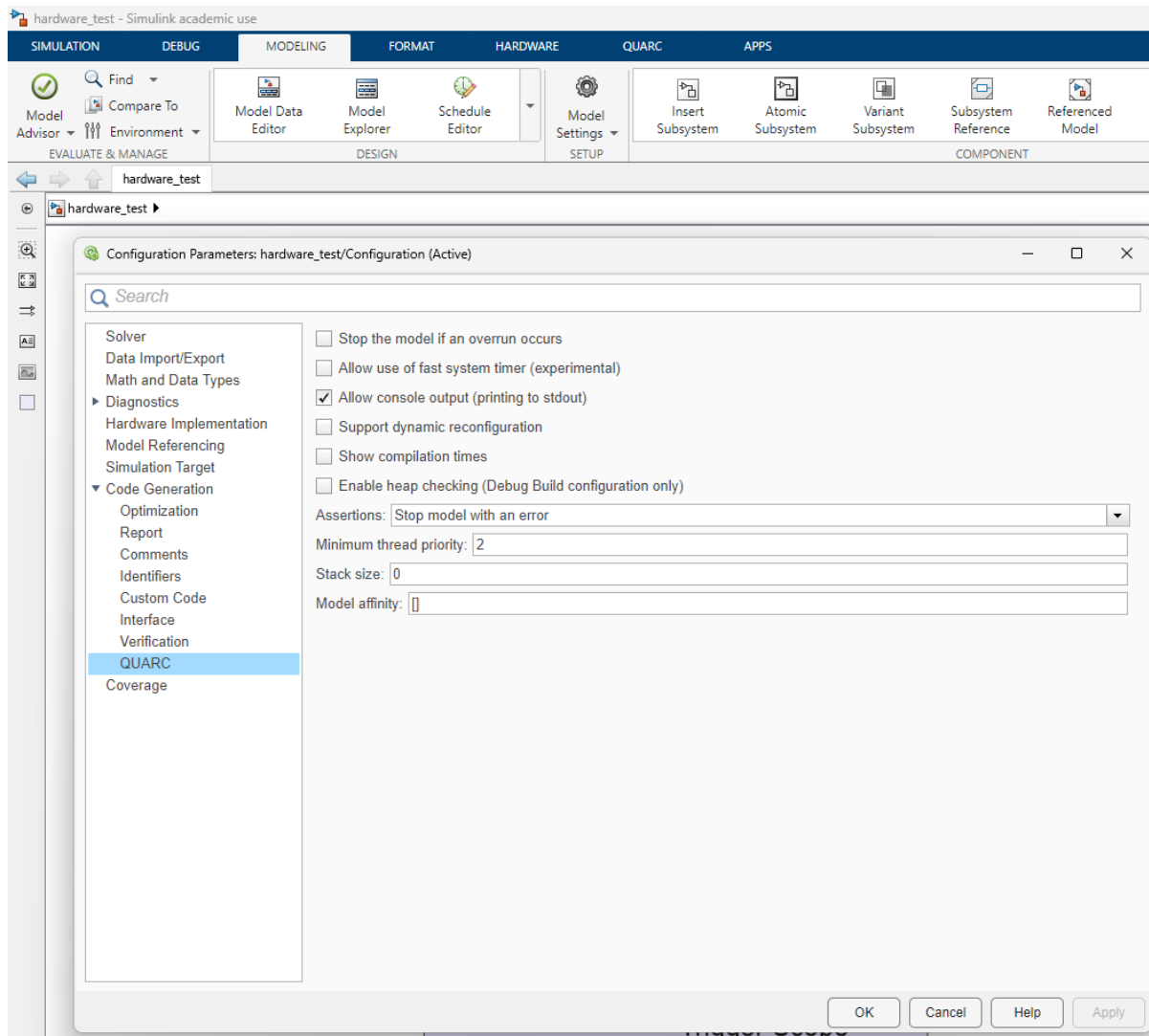
Right: hardware_test.mdl
This screenshot shows the Simulink model diagram. The model includes a "Test Signal" block, a "Motor Voltage" block, a "Motor Cart Plant" block, and a "Position" block. The "Motor Voltage" block is highlighted with a red box.

Text:
Set the HIL Initialize block to have board name HIL-1 and Board type q4. Click defaults to set up

Configuring model parameters for HIL & Quarc







Configuration Parameters: hardware_test/Configuration (Active)

Search

- Solver
- Data Import/Export
- Math and Data Types
- ▶ Diagnostics
- Hardware Implementation
- Model Referencing
- Simulation Target
- ▼ Code Generation
 - Optimization
 - Report
 - Comments
 - Identifiers
 - Custom Code
 - Interface
 - Verification
 - QUARC
 - Coverage

Simulation time

Start time: 0.0 Stop time: 10

Solver selection

Type: Fixed-step Solver: discrete (no continuous states)

▼ Solver details

Fixed-step size (fundamental sample time): 0.001

Tasking and sample time options

Periodic sample time constraint: Unconstrained

- ☒ Treat each discrete rate as a separate task
- ☐ Allow tasks to execute concurrently on target
- ☐ Automatically handle rate transition for data transfer
- ☐ Allow multiple tasks to access inputs and outputs
- ☒ Higher priority value indicates higher task priority

Important to have these set correctly
 $T_s = 0.001$, if you change it here change it everywhere...

OK Cancel Help Apply

Configuration Parameters: hardware_test/Configuration (Active)

Search

Solver

Data Import/Export

Math and Data Types

▶ Diagnostics

Hardware Implementation

Model Referencing

Simulation Target

▼ Code Generation

Optimization

Report

Comments

Identifiers

Custom Code

Interface

Verification

QUARC

Coverage

Hardware board: Determine by Code Generation system target file

Code Generation system target file: [quarc_win64.tlc](#)

Device vendor: IntelDevice type: x86-64 (Windows64)

▼ Device details

Number of bits

char: 8short: 16int: 32

long: 32long long: 64float: 32

double: 64native: 64pointer: 64

size_t: 64ptrdiff_t: 64

Largest atomic size

integer: Char

floating-point: Float

Byte ordering: Little EndianSigned integer division rounds to: Zero

☒ Shift right on a signed integer as arithmetic shift

☐ Support long long

...

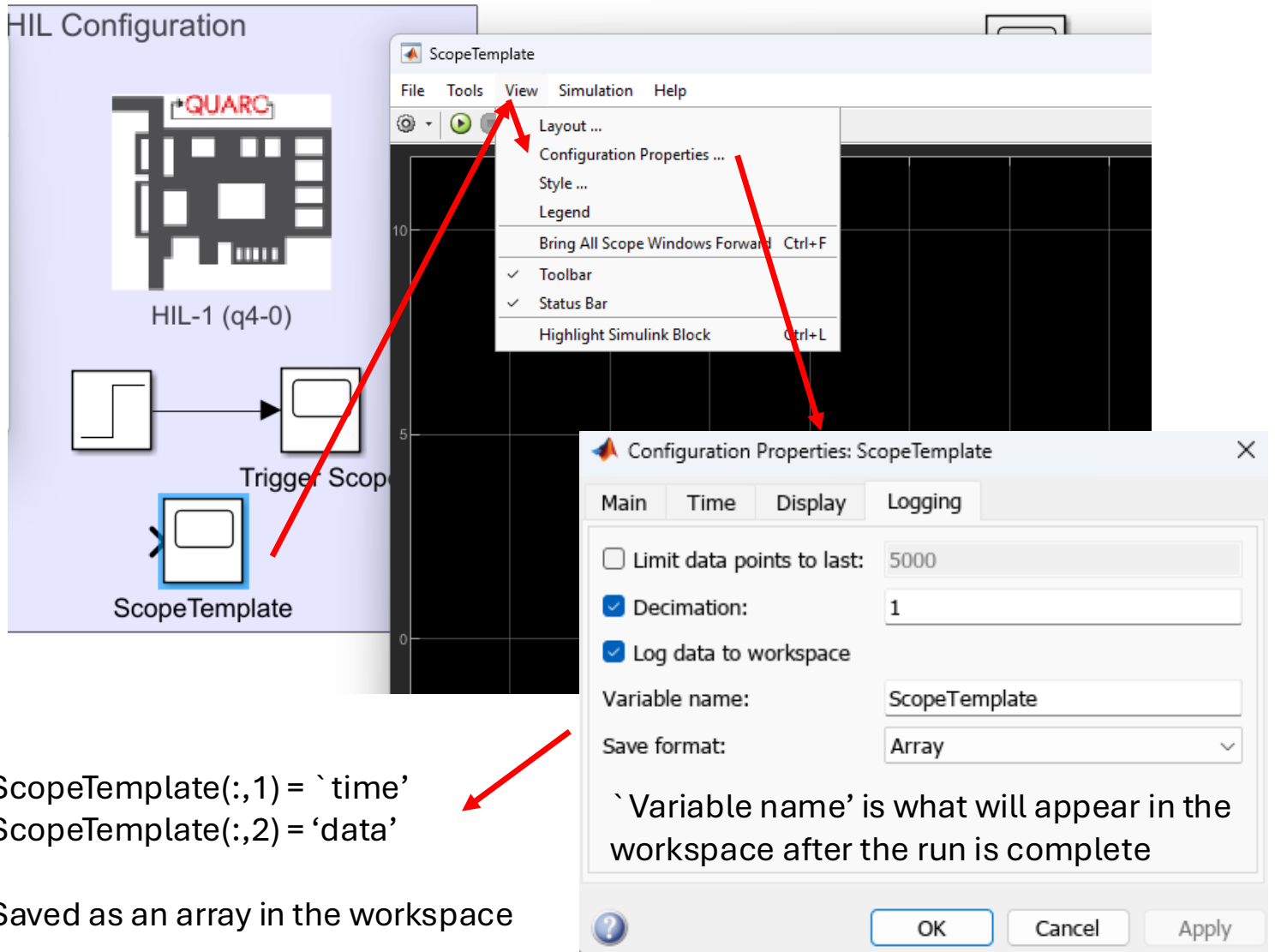
OK

Cancel

Help

Apply

Configuring scopes for data logging:



The screenshot shows the HIL Configuration window with a ScopeTemplate block highlighted. A red arrow points from the ScopeTemplate block to the Configuration Properties dialog. The dialog is open to the Logging tab, showing settings for data logging. The Variable name is set to ScopeTemplate and the Save format is set to Array. A red arrow points from the text below to the Variable name field.

ScopeTemplate(:,1) = `time`
ScopeTemplate(:,2) = `data`

Saved as an array in the workspace

Configuration Properties: ScopeTemplate

Main Time Display Logging

☐ Limit data points to last: 5000

☒ Decimation: 1

☒ Log data to workspace

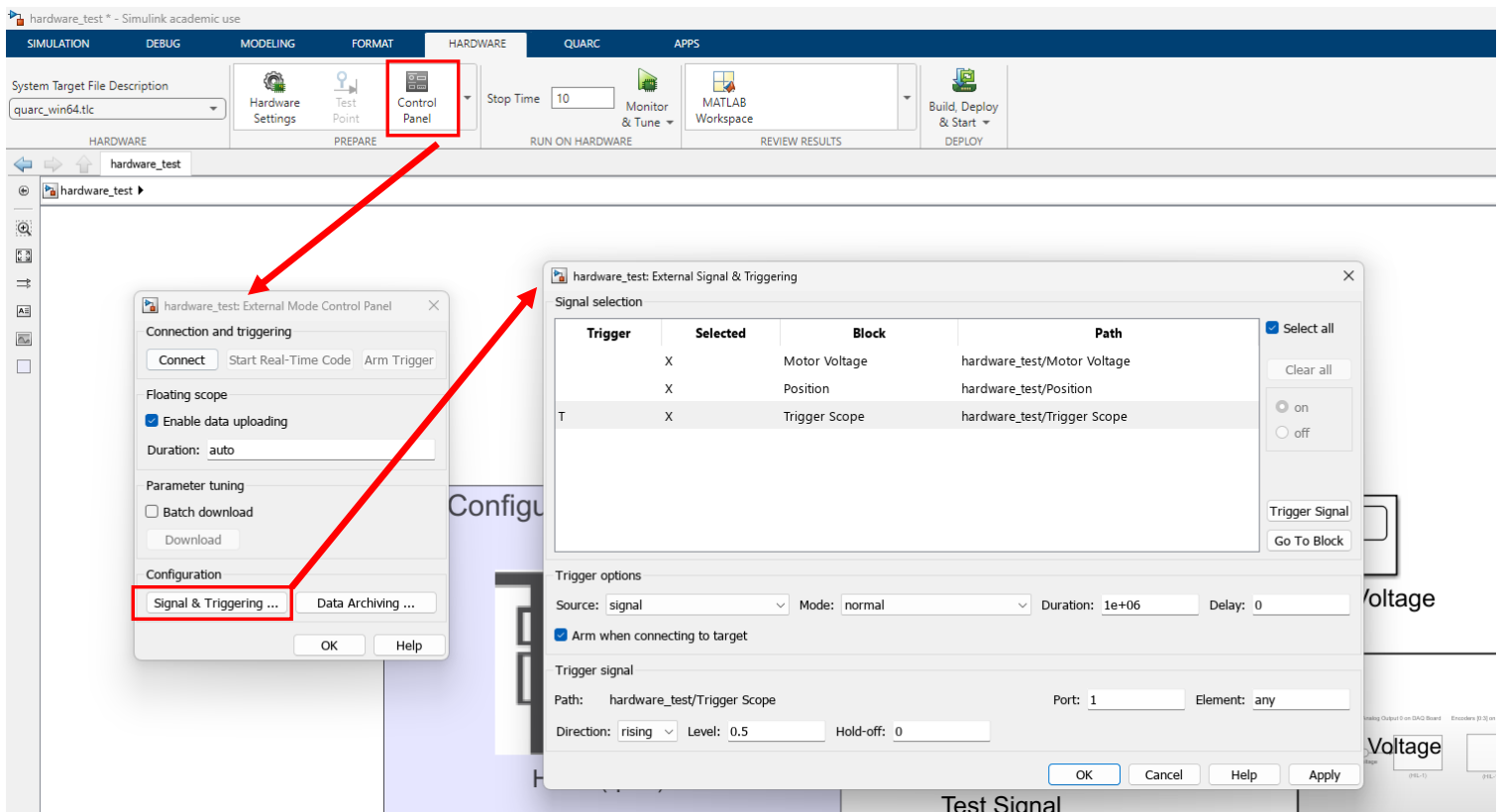
Variable name: ScopeTemplate

Save format: Array

`Variable name' is what will appear in the workspace after the run is complete

OK Cancel Apply

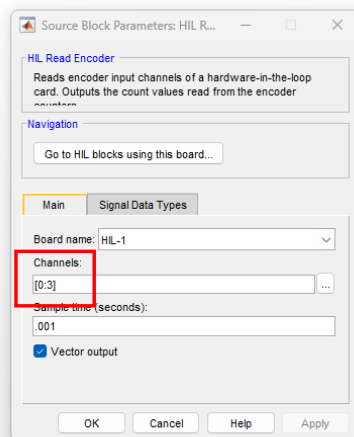
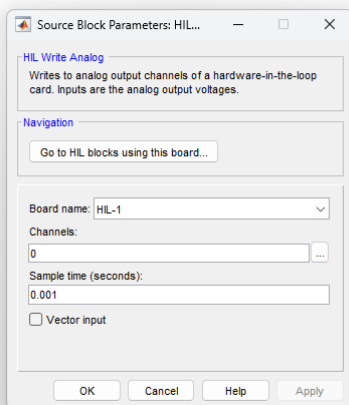
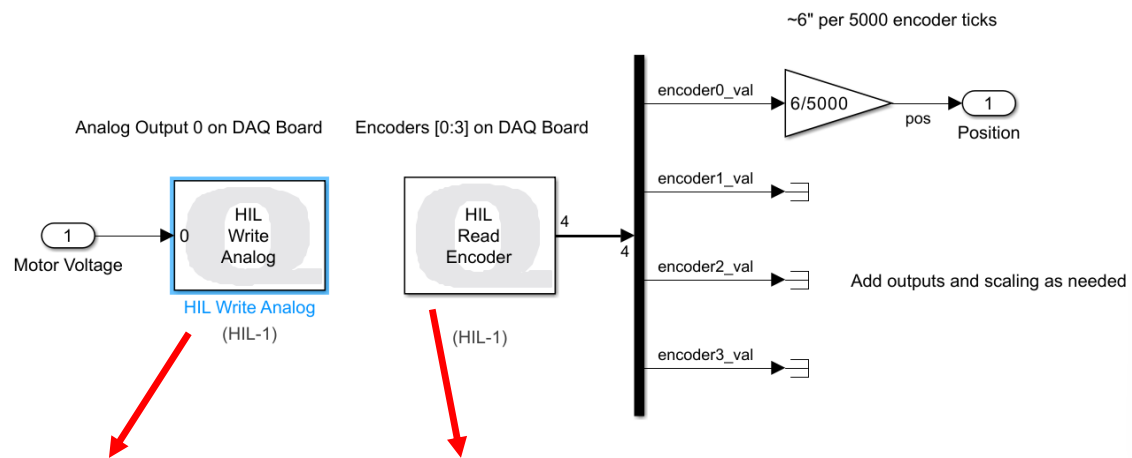
Configuring triggering and model data logging:



X under Selected means the signal will be logged

T under trigger means that will be used as trigger signal (don't change)

Duration is maximum # of samples logged, $Duration = t_{Stop}/T_s$



Demo model is reading all 4 encoders as vector and demuxing (shown above)

Can set to just read a single encode

Click 'Build' to make sure model can be compiled

The screenshot shows the Visual Studio IDE interface. In the top toolbar, the 'Build' button (represented by a hammer icon) is highlighted with a red box. Below the toolbar, the 'Diagnostic Viewer' window is open, displaying the 'Build Summary' for the 'hardware_test' project. The summary indicates that the model was successfully built.

Build Summary

Top model targets built:

Model	Action	Rebuild Reason
hardware_test	Code generated and compiled.	Generated code was out of date.

1 of 1 models built (0 models already up to date)
Build duration: 0h 0m 9.4402s

^Successful build message

View diagnostics

Monitor & Tune builds & runs the code.
Expected outputs shown

